ZISK WHITEPAPER[1]

ZISK CORE TEAM

Abstract. We present ZisK, a STARK-based zero-knowledge virtual machine (zkVM) that executes programs compiled from standard instruction set architectures and produces succinct, transparent proofs of their correct execution. ZisK targets the RISC-V ISA, allowing developers to use existing Rust and C toolchains without modification. Moreover, ZisK's modular architecture enables the addition of optimized constraint systems for expensive operations such as elliptic-curve arithmetic and bulk memory access, making it practical to prove complex computations at scale.

This paper provides a precise specification of the ZisK design: its instruction set architecture, the arithmetization framework, the circuit architecture and chip design, the continuation mechanism, and the recursive aggregation system.

Contents

1. Introduction **1**
2. Instruction Set Architecture **3**
3. Arithmetization Framework **9**
4. ZisK Design **14**
5. Continuations **22**
6. Recursion **25**
References **30**
Appendix **31**

1. Introduction

Recent advances in zero-knowledge (ZK) proving systems [GWC19], [Ben+18a], [DP23], [HLP24] have opened the door to a new class of cryptographic applications: systems that can verify the correct execution of an arbitrary computation without revealing its private inputs, in time and space sublinear in the original computation. These properties underpin a rapidly expanding class of applications in blockchain scalability—such as validity rollups, verifiable coprocessors, and trustless cross-chain bridges—as well as an increasingly broad range of off-chain computation protocols.

1.1. Background and Motivation.

Early ZK systems required developers to express computations directly as arithmetic circuits using frameworks such as Circom [Bel+23] or Halo2 [Ele22]. This low-level approach offers the greatest flexibility —developers can hand-tune constraint counts and choose their proof backend— but comes at a significant cost: every new computation requires custom circuit design, security review, and

[1]This document describes ZisK v0.16.1 [48cf7cc – 2026-20-03].
Source code available at `github.com/0xPolygonHermez/zisk`.

ongoing maintenance. The productivity model is fundamentally different from standard software development, limiting who can build ZK-enabled applications.

Zero-knowledge virtual machines (zkVMs) address this by interposing a fixed, general-purpose circuit between the developer and the proving system. The circuit verifies the correct execution of programs targeting a standard instruction set architecture (ISA), so developers can write programs in familiar languages like Rust or C and obtain proofs automatically, without writing a single constraint. The challenge for zkVM designers is then to make this general-purpose circuit as efficient as possible.

Existing zkVMs navigate a fundamental trade-off between developer experience and proving efficiency. Systems built around *ZK-native ISAs* (e.g., Cairo [GPR21] or Valida [Lit24]) define instruction sets whose semantics align naturally with finite-field arithmetic, enabling compact constraint representations but requiring developers to adopt a new programming model. Systems targeting *standard ISAs* such as RISC-V [RIS19] (e.g., RISC Zero [RIS23], SP1 [Suc24] or Nexus [Nex24]) preserve familiar toolchains and allow any RISC-V binary to run, but must constrain a general-purpose register machine whose operations — integer division, bitwise logic, memory addressing— are expensive to encode as polynomial constraints.

1.2. **ZisK.**

This paper introduces ZisK, a zkVM designed to make zero-knowledge proofs of general-purpose computation practical at production scale. ZisK targets the RISC-V ISA, preserving compatibility with mature toolchains such as LLVM and Rust so that programs can be proved without changes to existing development workflows. This positions ZisK in the standard-ISA camp, but with a deliberate architecture that treats proving performance as a first-class concern: rather than accepting the overhead of a general-purpose CPU circuit, ZisK uses a modular design where expensive operations are handled by dedicated, independently optimized constraint systems.

A central motivation for ZisK is *extensibility*. Application domains such as blockchain execution environments rely on a specific, known set of expensive primitives —elliptic-curve operations, hash functions, bulk memory transfers. A zkVM designed for these domains should be able to support those primitives natively, with proofs that are tight to the operation's intrinsic complexity. ZisK is structured so that such optimized components can be added as self-contained modules without modifying the rest of the system, making it practical to tailor the prover to a specific application's workload. Building on STARKs [Ben+18b] further ensures that this is achieved without a trusted setup and with post-quantum security.

1.3. **Paper Overview.**

Section 2 defines the ZisK ISA: execution environment, machine state, and instruction format. Section 3 presents the arithmetization framework: AIRs, bus

interactions, and LogUp-based circuit satisfiability. Section 4 describes the chip architecture and bus system. Section 5 explains trace splitting and how continuation buses chain segment boundaries. Section 6 covers the recursive aggregation tree, the recursive challenge derivation, and root-level bus balancing.

1.4. **Notation.**

For the remainder of the paper, we fix $\mathbb{F}$ to be a finite field of prime order p. In particular, we use the Goldilocks prime field $p = 2^{64} - 2^{32} + 1$, though this choice is not fundamental to the overall design and any appropriate prime would suffice. When needed, we identify field elements with integers in the range $[0, p)$.

We also fix a finite extension field $\mathbb{K}$ of $\mathbb{F}$, primarily for the security of the underlying proving system; concretely, ZisK targets 128-bit security. In particular, we use the third Goldilocks extension $\mathbb{F}[X]/(X^3 - X - 1)$, whose order $|\mathbb{K}| = p^3 \approx 2^{192}$ gives enough room to drive the proving system's soundness error well below 2^{-128}, while still allowing efficient arithmetic. Again, this choice is not fundamental to the design.

We denote by $\mathbb{F}[X]$ the ring of univariate polynomials over $\mathbb{F}$ and by $\mathbb{F}[X_1, ..., X_n]$ the ring of n-variate polynomials over $\mathbb{F}$. When needed, we write $\mathbb{F}^{<N}[X]$ to refer to the subset of polynomials of degree strictly less than N. We use $\mathbb{F}^{n\times m}$ to denote the set of n-by-m matrices over $\mathbb{F}$. The same notation applies to $\mathbb{K}$.

We use the notation $[a, b]$ to denote the set of integers from a to b, inclusive; and $[a, b)$ to denote the set of integers from a to b, inclusive of a and exclusive of b. When $a = 1$ and $b > 1$, we write $[b]$ instead of $[1, b]$.

2. Instruction Set Architecture

The ZisK instruction set architecture (ZisK ISA) specifies the computational model that ZisK proves correct: the execution model, the memory layout, the instruction format, and the full set of operations. Because the ZisK ISA is a *virtual* ISA — not targeting any physical hardware — its design can prioritize properties that simplify zero-knowledge proof generation rather than properties that suit a particular microarchitecture.

The ZisK ISA is designed around two goals. The first is compatibility: ZisK targets the RISC-V ISA so that programs written in standard languages and compiled with existing toolchains can be run without modification. The second goal is efficiency: operations that would be expensive to constrain step-by-step in the main execution circuit —elliptic-curve arithmetic, hash functions, bulk memory transfers— are exposed as *precompile instructions*, each handled by a dedicated, optimized sub-circuit. This combination gives the developer accessibility of a standard ISA with the proving performance of a custom one.

Remark (RISC-V to ZisK transpilation). Although programs are written and compiled as standard RISC-V binaries, ZisK does not constrain RISC-V execution directly. Before proving begins, the binary is *transpiled* off-chain into an equivalent program in the ZisK ISA, and it is the transpiled program that is committed to and executed by the VM. The transpilation is a deterministic, one-to-one rewriting from RISC-V instructions into one or more ZisK instructions; it is performed by a trusted off-chain tool and is not itself part of the proof. We therefore treat the transpiler as part of the trusted computing base.

The rest of this section describes the execution environment (Section 2.1), the machine state (Section 2.2), and the formal execution model (Section 2.3).

2.1. Execution Environment.

Programs in ZisK are executed inside a *virtual machine* (VM). The VM is a *state machine*: execution proceeds as a sequence of discrete steps, each step transforming the current *guest state* according to one instruction. The VM is hosted on a physical machine, called the *host*, that maintains an external *host state* separate from the guest. The ZisK ISA provides specific interfaces through which the guest program can read from and write to the host state.

This host/guest separation is central to ZisK's trust model. The guest program runs in a fully deterministic, sandboxed environment: every value it operates on is either derived from the initial program ROM (Section 2.2.1), loaded from memory, or explicitly received from the host through a typed channel. The host can supply:

- **Public inputs**: values known to both prover and verifier, loaded into the input data region before execution begins.
- **Private inputs** (hints): non-deterministic values provided by the host at runtime. The guest may request a hint and use its value, but must verify any properties it relies on —ZisK treats hints as unconstrained. A hint that is never verified proves nothing.

The key property of this model is that *everything the guest touches is constrained by the VM's arithmetization.* Memory accesses are constrained by the memory argument; program code is validated by the ROM bus on every fetch; and public outputs become explicit proof data. The host state, by contrast, lies outside and is trusted by convention.

2.2. State.

As defined before, the VM execution state is everything that changes from one step to the next. It is divided into the *guest state* —manipulated by the program — and the *host state* —maintained externally and supplied before or during execution. The guest state comprises the program ROM (read-only), the program counter `pc`, data memory (read/write), and the public outputs. The host state comprises the input stream and the hint stream.

2.2.1. *Program ROM.*

ZisK can execute any program compiled to its ISA. A program consists of two parts:

- **The Program Code**. The program code maps 32-bit instruction addresses to ZisK instructions. It is loaded into the VM before execution begins.
- **The ROM Data**. The ROM data is an immutable data segment holding program constants (e.g., string literals, lookup tables, and static data embedded in the binary). It occupies a dedicated region of the address space and can be read but never written by the program.

Both parts are determined entirely by the program binary: they are identical across every execution of that program, regardless of the inputs provided. Once the program is fixed, neither the code nor the ROM data can change — no instruction can modify them and they play no role in the per-step state transitions of the machine. This separation of code from mutable data is the classical *Harvard architecture*: code and data live in distinct address spaces and are governed by different access rules.

From the proving system's perspective, the two parts are committed to and validated by distinct mechanisms. The program code is organised as a fixed table of instructions indexed by their addresses. Before any execution begins, this table is hashed into a *program commitment* that is included in the verifier's context. At every step, the main circuit performs a table lookup against this committed table —retrieving the instruction at the current program counter and verifying it belongs to the ROM. No instruction can be substituted or forged without invalidating the commitment.

The ROM data is validated through a different mechanism. Rather than a separate lookup, it is treated as a region of *immutable memory* within a dedicated address space (Section 2.2.5). At initialization, the ROM data region is written once to establish its contents; thereafter only reads are permitted. From the running program's point of view, reads from ROM data are indistinguishable from reads from writable RAM — the immutability is a property enforced by the arithmetization, not by the ISA.

2.2.2. *Instruction Format.*

ZisK instructions are specified by:

- An *opcode* determining the operation to perform.
- A *source* for a first input (a), specifying how to obtain its value.
- A *source* for a second input (b), specifying how to obtain its value.
- A *store* for the output (c), specifying where to write the result.

Each instruction performs an operation over two input operands, producing an output and a *flag*:

$$(c, \text{flag}) = \text{op}(a, b)$$

where $a, b, c \in [0, 2^{64})$ and flag $\in \{0, 1\}$. In essence, each instruction is a single execution step that transforms two 64-bit inputs into a 64-bit output and a boolean flag. The full ZisK instruction set is described in detail in `zisk_ops.rs`.

2.2.3. *Phantom Instructions.*

The opcode determines the operation performed by the instruction. Most instructions correspond to a single provable opcode and generate a proof obligation for that operation. Some instructions, however, are *phantom instructions*: they leave the guest state unchanged — acting as no-ops from the VM's perspective — but may trigger side effects in the host state, such as advancing the input or hint streams or emitting debug information. Phantom instructions appear in the main execution trace but carry no proof obligation of their own.

Two notable cases:

- **Precompile dispatch**: Instructions that implement the calling convention for invoking precompiles —passing parameters, triggering the call, and retrieving results. What gets proven is the precompile computation itself.
- **Halting**: This instruction terminates the emulator immediately with an error condition. Its opcode cannot appear in a valid execution trace —if it is ever reached, proof generation fails. It is present in the ISA as a sentinel for invalid or unreachable code paths.

2.2.4. *Program Counter.*

The program counter `pc`, which is seen as a special purpose register, stores the address of the instruction currently being executed. After each instruction, `pc` advances to the next instruction address unless the instruction explicitly modifies control flow (jumps, branches).

2.2.5. *Data Memory.*

Data memory is a *random access memory (RAM)* which supports read and write operations. It is separate from the program ROM and is used for mutable state during execution. The ZisK ISA defines a 32-bit byte address space for data memory, but the native unit of access is a 64-bit word (8 bytes). Memory cells are aligned to 8-byte boundaries, so valid addresses are multiples of 8. ZisK supports sub-word accesses (1, 2, or 4 bytes) by decomposing them into aligned 8-byte operations (Section 4.2.2).

To support execution of programs with different data assumptions and properties within the same architecture, we separate data memory into distinct regions, each with its own access semantics. Table 1 summarises the address space layout and the semantics of each region.

Region	Address Range	Semantics
Input Data	`[0x4000_0000, 0x8000_0000)`	[1GB] Written once at initialization, read-only thereafter
ROM Data	`[0x8000_0000, 0x8800_0000)`	[128MB] Program constants, read-only
Writable RAM	`[0xA000_0000, 0xA001_0000)`	[64KB] System region: registers, UART and CSR registers
	`[0xA001_0000, 0xA003_0000)`	[128KB] Output data: public outputs written during execution
	`[0xA003_0000, 0xC000_0000)`	[496MB] Program memory: general-purpose read/write RAM

TABLE 1. Memory layout of the ZisK ISA. The address space is divided into three main regions: input data, ROM data, and writable RAM.

The 32 general-purpose registers $x_0, ..., x_{31}$ occupy the start of the system region, each spanning 8 bytes, following the RISC-V ABI [RIS24]. Register x_0 is hardwired to zero. From the ISA's perspective, registers are addressed in the same numerical address space as data memory: a load or store targeting the register region behaves exactly as if it targeted RAM, which keeps the ISA uniform and lets compiler-emitted code access registers through ordinary memory operations. Internally, however, ZisK handles the register region separately from the rest of memory for performance reasons; this distinction is invisible to the program but is described in Section 4.2.

2.2.6. *Public Outputs.*

The guest program can explicitly expose values as public outputs. The values are written internally by a dedicated instruction and become part of the proof's public data that allow the verifier to check that the program produced specific results.

2.2.7. *Inputs and Hints.*

The host state maintains two streams through which the outside world communicates with the guest. The *input stream* is a queue of public data loaded before

execution begins. The *hint stream* provides non-deterministic values at runtime —the guest may request a hint and act on its value, but must verify any property it relies on, since ZisK treats hints as unconstrained. Both streams are consumed by copying data into guest memory via explicit instructions; any value received from the host is untrusted until the guest has verified it.

2.3. **Execution Model.**

The execution model defines how the VM transitions between states, what constitutes a valid execution, and how the prover commits to the result. Since the program ROM is fixed before execution begins, the *guest state* at step t is defined as:

$$S_t = (\mathrm{pc}_t, c_t, \mathrm{flag}_t, \mathrm{mem}_t[])$$

comprising the program counter, the previous instruction's output and flag, and the data memory. The host state $H = (\mathrm{in}[], \mathrm{hint}[])$ is supplied externally. Execution proceeds in discrete steps according to the following rules:

Initial state. Execution begins at the entry point with memory initialised from the program binary: the ROM data region is populated with program constants, the input data region is loaded from the public host-supplied input stream, and all writable memory is zeroed. Formally:

$$S_0 = (\mathrm{pc}_0, 0, 0, \mathrm{mem}_0[])$$

where pc_0 is set to a designated boot address and $\mathrm{mem}_0[]$ reflects the pre-loaded memory state.

Transition function. At each step, the VM fetches the instruction at the current program counter from the committed ROM, executes it, and advances the guest state. The host supplies hint values on demand. The single-step transition is:

$$S_{t+1} = \mathrm{Step}(S_t, \mathrm{hint}_t[])$$

where $\mathrm{hint}_t[]$ are the hint values supplied by the host at step t. The function Step is fully determined by the ZisK ISA: it decodes the instruction at pc_t, evaluates the operation, updates $\mathrm{mem}_{t+1}[]$, and sets pc_{t+1} according to the instruction's control-flow semantics.

Terminal condition. A T-step execution is *successful* if every transition is valid and the final program counter reaches a designated exit address. Execution *fails* if any step violates the ISA's invariants —invalid opcode, out-of-bounds memory access, or an explicit halt instruction— or if the program counter fails to reach the exit address at the end of execution.

Public outputs. The public outputs are not a separate component of the state but a designated region of writable RAM (see Table 1). The guest writes to this region through ordinary memory-store instructions during execution, and once the run terminates successfully, the contents of that region are extracted from $\mathrm{mem}_T[]$ and exposed as the proof's public output. In other words, the output

buffer is filled in place over the course of execution rather than being accumulated as a separate stream at every step.

3. Arithmetization Framework

A proving system is naturally divided into two layers. The *frontend* takes a program and produces a structured mathematical representation — typically a system of polynomial constraints — that captures exactly what it means for the computation to be correct. The *backend* then takes that representation and produces a cryptographic proof of its satisfiability. The two layers are decoupled by design: a single frontend can target different backends, and improvements to either do not require redesigning the other.

ZisK's *arithmetization framework* is its frontend. The choice of arithmetization is consequential: it determines how circuits are structured, what operations are native, and how efficiently the backend can prove them. ZisK's arithmetization is designed to be modular and flexible, capturing a wide range of computations while optimizing for the patterns that arise most frequently in zkVM execution.

The frontend is based on a generalization of the *algebraic intermediate representation (AIR)* framework [Ben+18b], which specifies polynomial constraints over a witness matrix; coupled with arguments such as LogUp [Hab22], which extends AIR with logarithmic derivative-based lookups. A single AIR captures the behavior of one logical unit (e.g., the CPU, the ALU, or the memory). Since realistic computations involve multiple interacting units, we group AIRs together and define *interactions* between them —messages exchanged through shared communication channels called *buses*. This modular design has several advantages:

- **Separation of concerns**: Each AIR can be designed, implemented, and tested independently.
- **Composability**: AIRs can be reused across different computations or combined in different configurations.
- **Scalability**: Different AIRs can have different sizes, allowing resources to be allocated based on actual usage.
- **Backend independence**: The arithmetization can be paired with any compatible proving system.

The remainder of this section formalizes these concepts.

3.1. **AIRs.**

We generalize the standard AIR definition [Ben+18b] by augmenting it with *AIR values*: a fixed vector of field elements attached to the instance rather than indexed by row. This extension is non-standard but necessary to express global constants —such as public inputs or aggregation roots— as first-class citizens of the constraint system rather than as workarounds encoded in the trace.

Definition 1 (AIR and Witness). An *algebraic intermediate representation* (AIR) of width W, depth D, and V values is a set of *polynomials*:

$$\mathcal{A} = \{C_i\}_{i \in I} \subset \mathbb{F}\Big[X_1^{(0)}, ..., X_W^{(0)}, ..., X_1^{(D)}, ..., X_W^{(D)}, Y_1, ..., Y_V\Big]$$

where I is a finite index set. Polynomials C_i are typically known as *constraint polynomials* or simply *constraints*.

A *witness* for $\mathcal{A}$ is a pair $(\boldsymbol{T}, \boldsymbol{a})$ where $\boldsymbol{T} \in \mathbb{F}^{N \times W}$ (with N a power of two) is the *trace matrix* and $\boldsymbol{a} \in \mathbb{F}^V$ are the *AIR values*. The j-th column of $\boldsymbol{T}$, viewed as a mapping $T_j : [N] \to \mathbb{F}$, instantiates the family of shifted variables $X_j^{(0)}, ..., X_j^{(k)}$: at row r, the variable $X_j^{(k)}$ takes the value $T_j(r + k)$. The variable Y_j is instantiated by a_j, the same value at every row. The witness *satisfies* $\mathcal{A}$ if for every constraint $C \in \mathcal{A}$ and every row $r \in [N]$:

$$C(T_1(r), ..., T_W(r), ..., T_1(r + D), ..., T_W(r + D), a_1, ..., a_V) = 0$$

where row indices are taken modulo N.

Not all columns in a trace have the same lifecycle. Some are fully determined by the program before any execution begins; others depend on the specific inputs provided; and others are intermediate values the prover computes but need not reveal. Distinguishing these cases allows the proving system to avoid redundant work —a column that is the same across all executions of a program need only be committed once— and to be precise about what the verifier is entitled to see. Trace columns are therefore partitioned into three categories according to when they are determined and who can access them:

- **Fixed**: Determined at setup time, independent of any particular execution. Fixed columns are shared with the verifier as part of the AIR's description; when an AIR has fixed columns, the trace height N is also fixed at setup. Typical uses include lookup tables and instruction encodings.
- **Cached**: Computed by the prover and committed once, then reused across multiple executions of the same program. Cached columns capture data that depends on the program but not on its inputs —such as the ROM instruction trace— avoiding redundant recomputation.
- **Witness**: Computed fresh for each execution and not revealed to the verifier. These columns carry the execution-specific state: operation values, memory accesses, and all intermediate values produced during a run.

3.2. Interactions.

A single AIR is self-contained: its constraint polynomials relate columns within the same trace and cannot reference columns of another AIR. Yet realistic computations are inherently cross-cutting —the CPU must read values that the memory has written, and a hash precompile must receive inputs from and return outputs to the main execution trace. To express these relationships without merging all AIRs into one monolithic constraint system, we introduce *interac-*

tions: row-level messages that one AIR sends and another receives, mediated by a shared channel called a *bus*. An interaction specifies which values are communicated at each row and with what multiplicity; consistency across AIRs is then enforced by requiring that every bus is *balanced* —the total signed flow of messages sums to zero.

Definition 2 (Interaction). Let $\mathcal{A}$ be an AIR of width W, depth D, V values. An *interaction* of length ℓ for $\mathcal{A}$ is a tuple $(b, m, \boldsymbol{e})$ where:

- $b \in \mathbb{F}$ is a *bus identifier*.
- $m \in \mathbb{F}\left[X_1^{(0)}, ..., X_W^{(0)}, ..., X_1^{(D)}, ..., X_W^{(D)}, Y_1, ..., Y_V\right]$ is the *multiplicity* polynomial.
- $\boldsymbol{e} \in \mathbb{F}\left[X_1^{(0)}, ..., X_W^{(0)}, ..., X_1^{(D)}, ..., X_W^{(D)}, Y_1, ..., Y_V\right]^{\ell}$ are the *message* polynomials.

Given a witness $(\boldsymbol{T}, \boldsymbol{a})$ for the host AIR, the *evaluation* of the interaction at row r is obtained by substituting $X_j^{(k)} \leftarrow T_{j(r+k)}$ and $Y_j \leftarrow a_j$. We write $m(r)$ and $\boldsymbol{e}(r)$ for the resulting scalar and vector in $\mathbb{F}^{\ell}$, leaving the witness implicit when clear from context.

Each AIR can be augmented with a set of interactions. At each row, the interaction emits the message $\boldsymbol{e}(r)$ with signed multiplicity $m(r)$: a positive multiplicity means the AIR *sends* the message, a negative multiplicity means it *receives* it.

Interactions define the communication structure, but they do not by themselves specify what semantic relationship is being enforced. That is the role of *arguments*.

3.3. **Arguments.**

Arguments are the semantic tools used to specify what consistency must hold across bus interactions: a permutation argument asserts that two multisets are equal or a lookup argument asserts containment in a table. Each argument type is realized as a pair of interactions on a shared bus —one AIR sending with positive multiplicity, another receiving with negative multiplicity— so that bus balancing becomes equivalent to the desired relationship holding. A detailed description of the specific arguments used in ZisK is provided in Appendix A.

Bus balancing is therefore the unified consistency condition underlying all arguments. However, checking it naively —summing over every row of every AIR — requires the verifier to process the full witness, defeating succinctness. We use *LogUp* [Hab22] to reduce bus balancing to a small set of polynomial constraints that the backend can verify efficiently. The reduction proceeds in two steps: first, compress each multi-element message into a single field element; then, express the balancing sum as a polynomial identity checkable at a random location.

3.3.1. *LogUp Reduction.*

The key insight of LogUp is that multiset equality can be recast as a rational identity over a random challenge. Two multisets S and R (with multiplicities) are equal if and only if, for a *challenge* α:

$$\sum_{s \in S} \frac{1}{\alpha + s} = \sum_{r \in R} \frac{1}{\alpha + r}$$

Here, by *challenge* we refer to a random field element sent by the verifier after the prover has committed to its witness; because the prover cannot predict it before committing, the challenge breaks any attempt to craft a witness that satisfies the identity by accident. In the non-interactive setting, challenges are derived deterministically from the prover's commitments via a collision-resistant hash (Fiat-Shamir transform [FS87]), preserving the same guarantee without interaction.

To apply this idea to bus balancing, we first need to compress each multi-element message $\boldsymbol{e}(r)$ into a single field element.

Definition 3 (Compressed Interaction Value). Given an interaction $(b, m, \boldsymbol{e})$ of length ℓ and a challenge $\gamma \in \mathbb{K}$, the *compressed value* at row r is:

$$\hat{e}(r) = b + \sum_{j=1}^{\ell} \gamma^j \cdot e_j(r)$$

The bus identifier b is included to prevent cancellation across different buses.

After compression, LogUp defines an auxiliary column to account for the sum. For each AIR $\mathcal{A}_i$ with trace of length N_i and set of interactions $\mathcal{I}_i$, the contributions of all its interactions to the bus are accumulated row by row into a *partial sum* S_i, defined by the recurrence:

$$S_i(r) = \begin{cases} \displaystyle\sum_{(b,m,\boldsymbol{e}) \in \mathcal{I}_i} \frac{m(1)}{\alpha + \hat{e}(1)}, & \text{if } r = 1 \\ \displaystyle S_i(r-1) + \sum_{(b,m,\boldsymbol{e}) \in \mathcal{I}_i} \frac{m(r)}{\alpha + \hat{e}(r)}, & \text{if } r > 1 \end{cases}$$

The final value $s_i = S_i(N_i)$ is the AIR's total signed contribution to the bus. Global bus balancing then reduces to a single scalar constraint:

$$\sum_{i=1}^{k} s_i = 0$$

This constraint is enforced at the aggregation level: individual AIR proofs expose s_i as a public output, and the aggregation step verifies that they sum to zero (see Section 6).

3.4. **Circuits.**

Similar to [Ope25], we define a *circuit* as a collection of AIRs that, together with interactions, represents a complete computation.

Definition 4 (Circuit). A *circuit* is a collection of pairs $\mathcal{C} := \{(\mathcal{A}_i, \mathcal{I}_i)\}_{i=1}^{k}$, where each $\mathcal{A}_i$ is an AIR with its own parameters (W_i, D_i, V_i) and each $\mathcal{I}_i$ is a set of interactions for $\mathcal{A}_i$.

A *witness* for $\mathcal{C}$ is a collection of witnesses $\{(\boldsymbol{T}_i, \boldsymbol{a}_i)\}_{i=1}^{k}$ for each AIR $\mathcal{A}_i$, with $\boldsymbol{T}_i \in \mathbb{F}^{N_i \times W_i}$ and $\boldsymbol{a}_i \in \mathbb{F}^{V_i}$, where the trace heights N_i may differ across AIRs.

The fact that each AIR carries its *own* trace height N_i is a deliberate design choice and one of the main reasons for organising a computation as a circuit rather than as a single monolithic AIR. Different AIRs typically encode different units of work with different natural sizes —one AIR may scale with the length of the program being proved, another with the number of times a particular operation is invoked, and another with the size of a fixed lookup table— and there is no reason for these sizes to coincide. Forcing all AIRs to share a common height would either pad the cheap ones with empty rows or truncate the expensive ones below their needs, both of which translate directly into wasted prover work. Allowing per-AIR heights instead lets each AIR be sized to its true workload, so the prover only pays for what each unit actually does. This independence is also what later makes per-AIR mechanisms such as continuations (Section 5) possible without affecting the rest of the circuit.

A *bus* is a communication channel that connects all interactions sharing the same bus identifier b. Through the bus, different AIRs exchange data by sending and receiving interactions. The set of all bus identifiers in a circuit $\mathcal{C}$ is denoted $\mathcal{B} = \{b : \exists i, (b, m, \boldsymbol{e}) \in \mathcal{I}_i\}$.

The key consistency condition is *bus balancing*: the total flow of values across each bus must sum to zero when weighted by multiplicities.

Definition 5 (Bus Balancing). Given a circuit $\mathcal{C} = \{(\mathcal{A}_i, \mathcal{I}_i)\}_{i=1}^{k}$ with associated witnesses $\{(\boldsymbol{T}_i, \boldsymbol{a}_i)\}_{i=1}^{k}$, a bus $b \in \mathcal{B}$ is *balanced* if:

$$\sum_{i=1}^{k} \sum_{(b,m,\boldsymbol{e})\in\mathcal{I}_i} \sum_{r=1}^{N_i} m(r) \cdot \hat{e}(r) = 0,$$

where $\hat{e}(r)$ is the compressed interaction value for the interaction at row r from Definition 3.

Bus balancing ensures coherent communication: every message sent must be received with matching content, and no message can be invented or dropped. Together, local constraint satisfaction and global bus balancing give the full notion of circuit satisfiability: a circuit is satisfied when every AIR accepts its witness and every bus is balanced.

Definition 6 (Circuit Satisfiability). A circuit $\mathcal{C} = \{(\mathcal{A}_i, \mathcal{I}_i)\}_{i=1}^{k}$ is *satisfied* by witnesses $\{(\boldsymbol{T}_i, \boldsymbol{a}_i)\}$ if:

1. Each witness $(\boldsymbol{T}_i, \boldsymbol{a}_i)$ satisfies its corresponding AIR $\mathcal{A}_i$.
2. Every bus $b \in \mathcal{B}$ is balanced.

4. ZisK Design

We describe the circuit architecture that constitutes the ZisK zkVM. The previous sections established the building blocks: the ISA defines what programs can run (Section 2), and the arithmetization framework defines how computations are represented as polynomial constraints (Section 3). Here we describe how ZisK instantiates that framework into a concrete system.

Definition 1 (ZisK). Given the ZisK ISA, the *ZisK zkVM* is the collection of:

1. A host execution environment capable of executing every ZisK instruction.
2. A circuit $\mathcal{C}$ (as in Section 3) that constrains a finite sequence of guest state transitions to be valid with respect to a public commitment to a program ROM and an initial guest state. The circuit exposes the final guest public outputs as a public output.
3. Methods to generate the trace matrices from the guest state transitions necessary to generate a proof for $\mathcal{C}$.

ZisK's circuit supports only a finite sequence of state transitions, whereas program execution is a priori unbounded. Fixing a single large trace height upfront is impractical: the height would have to be chosen conservatively enough to accommodate every possible program, inflating proof costs even for short executions, while still failing for programs that exceed it. *Continuations* resolve this tension by organizing and partitioning the execution trace into fixed-height segments, each proved independently. We describe how ZisK implements this mechanism in Section 5.

4.1. Architecture Overview.

The ZisK circuit is organized into *chips*, where each chip encapsulates a single AIR together with the methods to generate its trace matrix. Chips fall into two categories:

- **Instruction executor**: Owns the execution logic for a subset of the ZisK ISA. Each instruction is handled by exactly one chip, though a chip may handle multiple related instructions.
- **Table**: Provides auxiliary functionality —lookup tables, range checks, or cached computation results— shared by one or more instruction executors.

Chips communicate exclusively through *system buses*: shared channels through which chips exchange messages. The set of system buses used in ZisK is summarized in Table 2. While there are globally shared buses, each chip may expose interfaces for communicating with other particular chips. This modular design means that new instructions can be supported by introducing a new chip and connecting it to the appropriate buses, without modifying any existing chip.

Bus	Purpose
Operation Bus	Routes operations requiring dedicated constraints from some chip to other specialized chips
ROM Bus	Verifies each instruction fetch against the committed program ROM
Memory Bus	Enforces read/write consistency across all memory accesses
Continuation Buses	Carries chip state across segment boundaries
Table Buses	Connects instruction executor chips to shared lookup tables

TABLE 2. Bus system in ZisK. Each bus serves a specific purpose in the architecture, enabling modular communication between chips and supporting the overall proof structure.

An instruction executor may delegate part of its computation to other chips via a bus, which may in turn query a table chip, and so on. This creates a layered dependency structure, illustrated in Figure 1: each chip proves only its own local constraints, and bus balancing composes the correctness arguments upward into a global proof.

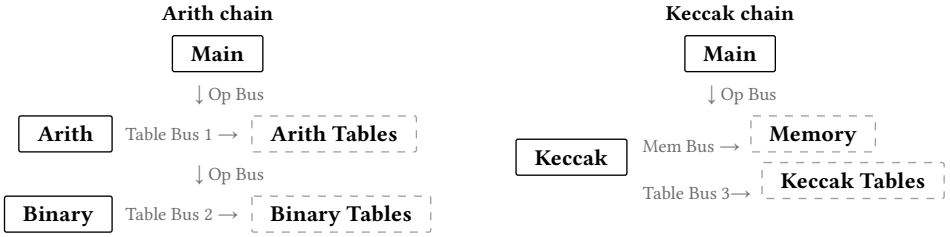


FIGURE 1. Two example chip dependency chains in ZisK. Solid borders denote instruction executor chips; dashed borders denote table chips. **Left**: the Main chip dispatches an arithmetic operation to Arith via the Operation Bus; Arith queries its own tables and further delegates to Binary, which queries its own tables. **Right**: Main dispatches a cryptographic operation to Keccak; the precompile simultaneously reads its inputs and writes its outputs via the Memory Bus and queries its own tables. In both cases each chip proves only its own local constraints, and bus balancing composes the arguments into a global proof.

Now we delve into the details of the design of the ZisK chips (Section 4.2) and bus system (Section 4.3).

4.2. **Chip Design.**

We now turn to the chips themselves: the AIRs that, together with the bus system, implement the ZisK zkVM. The presentation roughly follows the dependency structure of Figure 1, starting from the central spine and moving outwards. We begin with the *Main* chip (Section 4.2.1), which materializes one row per instruction step and orchestrates the rest of the system: it fetches instructions, resolves operands, dispatches operations to specialized sub-chips, and writes back results. We then describe the *Memory* chip (Section 4.2.2), which centralizes the memory consistency argument that almost every other chip ultimately relies on. The remaining categories cover what the Main delegates: *base-operation* chips (Section 4.2.3) handle classes of simple operations —binary, arithmetic, comparisons— through dedicated AIRs and lookup tables; *precompiles* (Section 4.2.4) handle expensive or memory-intensive primitives such as elliptic-curve arithmetic, hash functions, and bulk transfers, each as an independent instruction executor with its own internal layout; and *frequent-operation caches* (Section 4.2.5) commit pre-computed results for the most commonly queried inputs of base operations, replacing constraint checking with a direct lookup wherever possible.

4.2.1. *The Main.*

ZisK uses a *Main* chip as the central execution spine. Each row of the Main AIR corresponds to one instruction step and its trace materializes the full execution transcript. Recalling the notation from Section 2.2.2, the Main chip is responsible for:

1. **Instruction fetch**: At each step, the Main chip reads the instruction at the current program counter from the program ROM via the Program Bus.
2. **Operand resolution**: The input operands a and b are resolved according to their source modes (immediate, memory, or indirect), with memory reads constrained through the Memory Bus.
3. **Operation dispatch**: Operations that are simple enough are computed directly in the Main chip (e.g., no-ops, jumps, and control flow). Operations requiring dedicated constraint systems —binary operations, multiplications, cryptographic primitives— are *delegated*: the Main chip sends the inputs and receives back the result (c, flag) via the Operation Bus, and a specialized sub-chip proves correctness.
4. **State update**: The output c is written to its destination via the Memory Bus or directly in the Main trace.

4.2.1.1. *Registers.*

Although the ZisK ISA addresses the 32 general-purpose registers within the same numerical address space as data memory (Section 2.2.5), ZisK does *not* route register accesses through memory. Forcing every register read and write to traverse the Memory Bus would inflate the global memory trace —registers are by far the most frequently accessed locations in any execution— and would couple

the cost of register access to the cost of generic RAM. Instead, the register region is carved out of memory's domain and handled internally by the Main chip. Operand-resolution and store-back constraints reference register values directly from the Main chip, with a per-register linked-step argument (analogous to the global memory consistency argument, but local to the Main) ensuring that every register read observes the most recent write. From the program's standpoint nothing changes —a load or store to the register region behaves exactly as the ISA prescribes— but the underlying argument is local to Main rather than dispatched on the Memory Bus.

4.2.1.2. *Timestamps.*

The circuit arithmetization does not inherently represent temporal order. On the other hand, the execution of instructions is inherently sequential. To encode it, ZisK discretizes time by attaching a *timestamp* to every memory access, so that the memory consistency argument can verify that reads always observe the most recent write to each address.

Time is organized at two granularities. A *main step* t indexes one full instruction execution by the Main. Because rows within a trace are strictly increasing, t is a globally monotone counter across the entire execution; i.e., if $t_i < t_j$ for all $i < j$ and the instruction at step t_i is always executed before the instruction at step t_j.

A single main step, however, is not atomic from the memory's perspective: operand resolution requires reads, storing the result requires a write, and a dispatched precompile may independently access memory to read its inputs and write its outputs. To order all of these accesses, each main step t is subdivided into four fixed *memory step* timestamps:

$$t_{\text{mem}} = 4 \cdot t + s, \quad s \in \{0, 1, 2, 3\}$$

with slots $0, 1$ reserved for reads and slots $2, 3$ for writes. These four timestamps are the only ordering device: the address accessed at each slot is unconstrained and varies freely from instruction to instruction and from chip to chip. Any operation dispatched at step t can use the same four memory step values to stamp its own memory accesses —at addresses entirely different from those of the Main chip, but it can perform as much accesses as needed during its execution.

4.2.2. *Memory.*

The *Memory* chip centralizes all memory access verification. The Memory chip collects all submitted operations, verifies their consistency, and rejects any execution in which a memory access violates the memory rules.

To enforce correctness, all submitted operations are sorted globally by address, then by step within the same address. Within a given address, memory steps act as timestamps that must be strictly increasing; each time the address changes, the new address must be strictly greater than the previous one, so no address can appear more than once in non-consecutive blocks. This reduces the global

consistency requirement —that every read observes the most recent prior write — to a sequence of local, row-to-row checks.

Recall from Section 2.2.5 that address space is partitioned into regions, each governed by one of three access policies. The mutable RAM region supports arbitrary reads and writes: any address may be written at any step, and subsequent reads return the last written value; a read to an address that has never been written returns zero. Immutable regions (ROM and input data) admit exactly one write per address, which must be the first access to it; all later accesses are reads returning that same value.

The input data is also considered the region for the hint stream (Section 2.2.7): they carry no writes, their values are treated as unconstrained inputs supplied by the executor, and all accesses to the same address must agree on that value. Notably, the same address may be read multiple times at the same step in these free-input regions (i.e., the hint stream region described above) —a property exploited by precompiles that need to read the same input data repeatedly during their execution.

Value continuity for mutable memory follows from the sorted order: because rows for the same address are contiguous and ordered by step, the chip verifies each read against the immediately preceding row using adjacent-row constraints. Writes are additionally subject to a silent write prohibition: every write must produce a message on the Memory Bus, ensuring no memory modification can occur outside the scope of the consistency argument.

4.2.2.1. *Memory Access Paths.*

The access path for a memory operation depends on its width and alignment, as shown in Figure 2:

- **Aligned 8-byte access**: The requesting chip places the operation directly on the Memory Bus. The Memory chip processes it without further decomposition.
- **Sub-word or unaligned access** (1, 2, or 4 bytes): The requesting chip routes the operation to the *MemAlign* chip instead. MemAlign identifies whether the access falls within a single aligned 8-byte word or spans two consecutive words, decomposes it accordingly, and issues one or two aligned 8-byte operations to the Memory Bus.

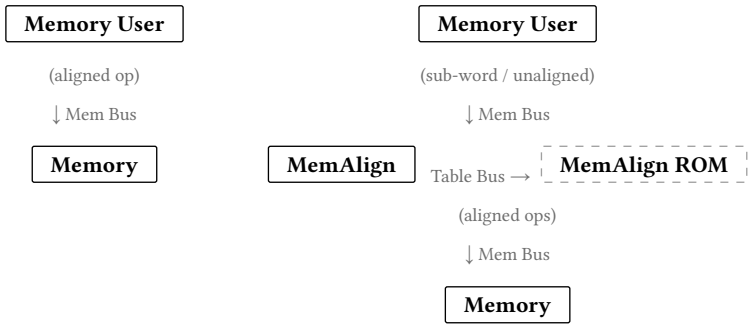


FIGURE 2. Memory access paths. Aligned 8-byte operations go directly to Memory; sub-word and unaligned operations are first decomposed by MemAlign into aligned 8-byte operations.

4.2.2.2. *MemAlign.*

The MemAlign chip models a small processor (the MemAlign ROM) with eight byte-wide registers that hold one aligned 8-byte word, and executes one of four fixed subprograms depending on whether the access is a read or write and whether it fits within one aligned word or spans two:

1. **Read within one word** [One read]: MemAlign performs one aligned read to load the surrounding 8-byte word into its registers, then extracts the requested bytes and returns the value.
2. **Write within one word** [One read, One write]: MemAlign reads the surrounding word, patches the relevant byte positions with the new value, writes the modified word back, and confirms the result.
3. **Read spanning two words** [Two reads]: The requested bytes straddle an 8-byte boundary. MemAlign reads both words, combines the relevant bytes from each, and returns the assembled value.
4. **Write spanning two words** [Two reads, Two writes]: MemAlign reads both words, patches the relevant bytes in each, and writes both modified words back.

In every case, MemAlign issues only aligned 8-byte operations to the Memory Bus. This design allows MemAlign to support all possible sub-word and unaligned accesses with a small, fixed set of constraints, without needing to know the specific access patterns in advance.

4.2.3. *Base Operations.*

The *Base Operations* chips prove the base instruction set operations —bitwise logic, comparisons, shifts, sign extensions, and integer multiplication and division— delegated from the Main chip via the Operation Bus.

These chips share a common structural profile that distinguishes them from the Main chip and from precompiles. First, they have no interaction with the Memory Bus: their inputs and outputs flow entirely through the Operation Bus, so they neither read from nor write to the memory state. Second, they carry

no state across segment boundaries and therefore use no continuation bus; each segment's proof is self-contained. These two properties mean that Base Operations chips are the simplest kind of instruction executor in ZisK.

Base Operations chips together with the Main chip and Memory chip form the core execution engine of ZisK, responsible for executing every instruction in the ZisK ISA. These chips are not designed for any particular class of operations; rather, they are designed to be flexible enough to support the full range of operations in the ISA, and their constraints are structured to allow efficient delegation of complex operations to specialized sub-chips.

Precompiles, on the other hand, are designed to handle complex operations that would be expensive to constrain step-by-step in the Main chip.

4.2.4. *Precompiles.*

Precompiles handle classes of operations —elliptic curve arithmetic, cryptographic hash functions, bulk memory transfers— whose decomposition into base operations would be inefficient. Each precompile is an independent instruction executor chip that owns the full proof obligation for its operation.

What distinguishes precompiles from basic operation chips is the complexity of their interface with the rest of the system. A precompile typically reads its inputs and writes its outputs directly from the memory state through the Memory Bus, rather than receiving them as simple 64-bit operands on the Operation Bus. Its computation may span many internal rows —for example, a hash permutation iterates over multiple rounds— so its trace height is sized to the operation's intrinsic complexity, independently of the Main chip. Some precompiles (such as DMA) even carry their own internal state across segment boundaries via a continuation bus.

At the ISA level, invoking a precompile follows a calling convention: the caller issues a sequence of parameter-passing instructions, a call instruction, and result-retrieval instructions. These steps appear in the Main chip trace as bookkeeping steps that set up and tear down the call, but carry no proof obligation themselves. Adding a new precompile requires only defining a new chip and wiring it to the appropriate buses, with no changes to any existing chip.

4.2.5. *Frequent-Operation Caches.*

Basic operation chips carry a fixed per-invocation overhead: each chip handles a whole family of operations, and a single query pays for the constraints that distinguish all of them, regardless of which one is actually being proved. Across a typical program execution, however, only a small fraction of all possible input combinations actually appear, and those that do appear tend to repeat. ZisK exploits this with *frequent-operation caches*: for binary and arithmetic operations, results for the most commonly queried input pairs are pre-computed and committed once at setup time.

At proof time, sub-chips resolve queries directly from the cache for inputs it covers, and fall back to constraint checking for the remainder. This significantly reduces witness size and proving time for programs with typical workloads.

4.3. **System Buses.**

ZisK uses a bus system —recall Table 2 for an overview— to constrain valid state transitions during program execution. Now we discuss all of them except the continuation bus, which is described in Section 5.3.

4.3.1. *Operation Bus.*

The *operation bus* routes complex operations from the Main chip to specialized sub-chips. Messages have the form:

$$(\mathrm{op}, a, b, c, \mathrm{flag}, t, \mathrm{extra}),$$

where $\mathrm{op} \in [0, 2^8)$ identifies the operation, $a, b \in [0, 2^{64})$ are the input operands, $c \in [0, 2^{64})$ is the output, $\mathrm{flag} \in \{0, 1\}$ indicates whether the operation was valid, t is the main step (Section 4.2.1.2), and $\mathrm{extra} \in [0, 2^{32})$ carries any additional information needed to prove correctness.

The main step t plays two different roles depending on the kind of sub-chip on the receiving end. For sub-chips whose entire input/output is already conveyed by (a, b, c) —i.e. those that do not interact with memory— t is set to 0 and carries no information: the operation is fully specified by its operands. For precompiles, by contrast, (a, b, c) is not enough: precompiles read their inputs and write their outputs directly through the memory bus rather than through the operand triple, so each precompile invocation must be tied to a specific point in the execution timeline. In that case, t is set to the current execution step of the Main chip, allowing the precompile's memory accesses to be ordered consistently with the rest of the trace.

When the Main chip delegates an operation, it sends a message on the operation bus and asserts that $(c, \mathrm{flag}) = \mathrm{op}(a, b)$ without verifying the operation locally. The corresponding sub-chip receives those messages and proves correctness through its own constraints.

4.3.2. *ROM Bus.*

The *ROM bus* ensures correctness of instruction fetching. Messages have the form:

$$(\mathrm{pc}, \mathrm{a_imm}, \mathrm{b_imm}, \mathrm{op}, \mathrm{store_offset}, \mathrm{jmp_offsets}, \mathrm{flags}),$$

where $\mathrm{pc} \in [0, 2^{32})$ is the program counter, $\mathrm{a_imm}, \mathrm{b_imm} \in [0, 2^{64})$ are the immediate values, $\mathrm{op} \in [0, 2^8)$ identifies the operation, $\mathrm{store_offset} \in [-2^{32}, 2^{32})$ is the memory offset for store instructions, $\mathrm{jmp_offsets} \in [-2^{32}, 2^{32})^2$ are the jump offsets for control flow instructions, and $\mathrm{flags} \in \{0, 1\}^k$ are flags indicating various instruction properties (e.g., whether it is a precompile call).

Instruction executor chips query this bus to verify that the instruction they execute exists in the program ROM at the given program counter. A dedicated ROM chip provides the program ROM as a lookup table.

4.3.3. *Memory Bus.*

The *memory bus* is the channel through which every chip that reads from or writes to an address space submits its memory operations. Messages have the form:

$$(\mathrm{op}_{\mathrm{mem}}, \mathrm{addr}, t_{\mathrm{mem}}, \mathrm{width}, \mathrm{data}),$$

where $\mathrm{op}_{\mathrm{mem}} \in \{0, 1\}$ is the access type (0: read, 1: write), $\mathrm{addr} \in [0, 2^{32})$ is the target address, t_{mem} is the memory step (Section 4.2.2), $\mathrm{width} \in \{1, 2, 4, 8\}$ is the access width in bytes, and $\mathrm{data} \in [0, 2^{8 \cdot \mathrm{width}})$ is the value read or written. The Memory chip collects all messages on this bus and enforces global memory consistency.

5. Continuations

A single chip proof covers a trace of fixed height N. Real programs run for an a priori unbounded number of steps, so a single proof cannot certify an arbitrary execution. *Continuations* are the mechanism by which a zkVM extends beyond this bound: the execution is divided into *segments*, each covered by an independent proof, and the proofs are linked into a single argument for the full run.

5.1. **The Standard Approach to Continuations.**

The most common continuation strategy splits execution into *checkpointed* segments. Execution runs for N steps, halts, and the prover generates a proof for those N steps; the machine state at the halt —the *checkpoint*— is saved. A fresh execution then resumes from that checkpoint and runs for another N steps, and so on. Each segment proof certifies one N-step block, and consecutive proofs are chained by having each segment verify the preceding checkpoint as part of its own public inputs.

This approach carries a significant overhead: *padding*. If a chip performs only $k \ll N$ operations in a given segment, its $N - k$ remaining rows must be filled with dummy entries that satisfy the constraints but carry no useful computation. In a multi-chip system, this cost compounds across every chip in every segment: different chips have wildly different utilization across segments, yet each padded row still enters the polynomial commitment and constraint evaluation. The result is a large blowup in prover time and proof size, which grows with the number of segments and the number of chips.

A solution at the cryptographic layer is the *jagged polynomial commitment scheme (Jagged PCS)* [Hem+25]. This protocol commits to the full computation trace as a single polynomial over a jagged matrix —where each column corresponds to a chip and can have a different height— and lets the verifier emulate access to the individual per-chip polynomials. The prover pays proving cost

proportional to the actual trace area rather than the padded one. The jagged PCS is, however, inherently multilinear: it is built on multilinear polynomial commitments and can be incorporated efficiently only into proving systems whose arithmetization is multilinear-based.

Rather than developing new cryptographic infrastructure and tying the solution to a specific class of proving system, ZisK addresses the problem at the system design level, described next.

5.2. **ZisK's Approach: Trace Splitting.**

ZisK decouples segmentation from execution and lets chips operate on demand. The program runs to completion in the host execution environment in a single pass, producing the full execution trace. Each chip records only the rows for instructions it handles: the Memory chip records only memory accesses, Keccak only hash operations, and so on. Chips that are rarely invoked produce short traces; chips that are heavily used produce long ones.

The prover then partitions each chip's trace independently into consecutive segments of that chip's fixed height N_i. Padding is limited to the final segment of each chip, where the last partial slice is padded to N_i rows; all earlier segments are filled to capacity. This stands in contrast to the standard approach, where every chip is padded to its height in every execution segment regardless of how many operations it actually performed. Figure 3 illustrates this layout.

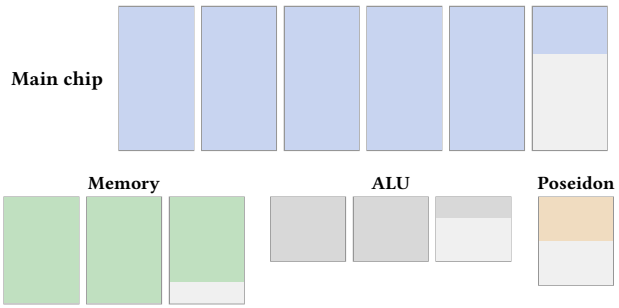


FIGURE 3. Padding layout for ZisK's trace-splitting approach. Chips define their own segment height N_i. Colored rows hold actual computation; gray rows are padding. Only the final segment of each chip requires padding.

Trace splitting eliminates most padding —in fact, it is equivalent to the Jagged PCS approach— but it introduces two problems that must be solved.

The first is *cross-chip consistency*. As commented in Section 4, there are chips that delegate operations to other specialized chips and record the expected result. Proving that the delegation was correct requires that the involved chips agree on the exchanged values. In the standard continuation's solution this agreement is enforced directly by generating a single monolithic proof of the entire trace. In the trace-splitting design the proofs for different chips are generated independently. Binding them into a globally consistent argument requires a shared

challenge that spans all chip instances —a check that cannot be performed locally and is instead delegated to the recursive aggregation layer. We explain the solution to this problem in Section 6.

The second is *cross-segment state*. Because each chip's trace is divided independently, a chip whose state evolves over the course of execution cannot restart cold at the beginning of each segment: consecutive segment instances must agree on the machine state at their shared boundary. This is the problem that continuations, in the classical sense, solve.

For chips whose computation is self-contained within each segment —such as base operations— the cross-segment state is empty: consecutive segment instances are independent and carry no shared context. For chips with persistent state, however, consecutive instances must agree on the state at their shared boundary. This shared boundary state is enforced by the *continuation bus*.

5.3. **The Continuation Bus.**

The *continuation bus* carries cross-segment state for each chip that requires it. Each segment instance sends its final state as a message on the bus and receives the initial state expected by the next instance. Bus balancing (Section 3.4) ensures the chain is globally consistent: every sent message is received exactly once with matching content, so gaps, duplications, and reorderings are all ruled out.

The state payload is chip-specific. The Main chip carries the program counter pc of the first instruction in the next segment and the last c value, so the next segment begins from exactly the state where the previous one left off. The Memory chip carries the last accessed address and timestamp, preserving the global chronological ordering of memory accesses across boundaries. Every chip with state that must survive a segment boundary defines its own continuation bus with the appropriate payload.

The *genesis* anchor injects the initial state assumed by the first segment instance; the *terminus* anchor receives the final handoff and checks that execution ended at the designated program counter. A global check ensures that each anchor fires exactly once, ruling out invalid segment chaining.

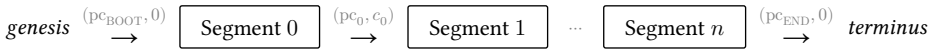


FIGURE 4. Continuation bus for the Main chip. The genesis anchor injects the boot program counter; the terminus anchor receives the final handoff at the designated end program counter.

5.4. **Parallel Proving and Recursive Aggregation.**

Because each segment instance is proved independently, all segments across all chips can be proved in parallel. Each individual proof establishes only *local consistency*: it certifies that the segment's own constraints are satisfied and records its contribution to the shared bus interactions as a partial sum. Two global checks remain outstanding.

First, the continuation buses must balance globally. Every cross-segment state message sent by a segment instance must be received exactly once by the next, with matching content. No individual segment proof can verify this —it requires aggregating the partial sums from all instances of a chip across all of its segments.

Second, the cross-chip buses must balance globally. The partial sums recording interactions between the Main chip and the Memory, or other specialized chips must cancel to zero when combined. This is the cross-chip consistency problem introduced earlier, and it likewise cannot be enforced locally.

Both checks reduce to a single global field equation: the sum of all partial sums across all instances and all chips must equal zero. The recursive aggregation layer (Section 6) accumulates these partial sums up an aggregation tree and enforces this equation at the root, binding the independent segment proofs into a single, globally valid argument for the full unbounded execution.

6. Recursion

ZisK uses proof recursion to combine the independent per-segment proofs produced by trace splitting into a single, globally consistent argument. This section introduces proof recursion in general, identifies the two specific problems it resolves in ZisK, and details the aggregation structure.

6.1. **Proof Recursion.**

A proving system is called *recursive* when it can verify the validity of another proof as part of its own computation. The key observation is that a verifier is itself a computation —it reads a proof and performs arithmetic checks— so the verifier itself can be expressed as polynomial constraints and be proved. A *recursive verifier* is therefore an arithmetic circuit that takes a proof π and its public inputs as witness and checks that π is valid. Wrapping this circuit in a proof yields a *recursive proof*: a succinct argument that attests to the correctness of an underlying proof without re-exposing its content.

The power of this construction comes from its composability. Since a recursive proof is itself a valid proof, it can in turn be fed into another recursive verifier, producing a proof of a proof of a proof. Iterating this idea allows a sequence of computations —or a collection of independent computations— to be folded into a single, constant-size argument regardless of the number of steps or the size of the individual proofs. The result is a *proof composition* that scales arbitrarily: adding more computations does not make the final proof larger or more expensive to verify.

Recursive composition serves two purposes. The first is *proof aggregation*: combining many independent proofs $\pi_1, \ldots, \pi_k$ into a single argument that attests to the validity of all of them simultaneously. The second is *proof compression*: converting a proof generated with one proving system (e.g., a proof with large size but fast to generate) into a proof in another system better suited for a particular context (e.g., a compact proof with fast verification).

6.2. **The Role of Recursion in ZisK.**

Recall from Section 5 the two open problems left by trace splitting.

The first is *cross-chip bus balancing*. Each segment proof contributes a partial sum $s_i \in \mathbb{K}$ to the global bus interaction. Global correctness requires $\sum_i s_i = 0$, but no individual proof can verify this: it requires aggregating the partial sums across all segment instances and all chips.

The second, and more subtle, is *challenge derivation*. The LogUp argument (Section 3) uses random field elements α and γ, derived via the Fiat-Shamir heuristic, to compress bus interactions into a single accumulated value per AIR. For the global balancing equation $\sum_i s_i = 0$ to be meaningful, all partial sums must be computed under the *same* challenges. In a monolithic proof this is automatic: there is one prover transcript and one application of Fiat-Shamir. In the trace-splitting design, each segment is proved independently and would naturally derive its own challenges from its own commitments —producing values that differ across instances and making the partial sums incommensurable.

Both problems are resolved by the recursive aggregation layer.

6.3. **Challenge Derivation.**

The challenge derivation problem arises because Fiat-Shamir ties each challenge to the transcript of a single prover. When segment provers operate independently, their transcripts diverge and so do their challenges.

The solution is to separate challenge derivation from proof generation: all segment provers first commit to their traces, and the challenge is then derived from the full set of commitments and broadcast to all provers as a *public input*. Each segment proof proves the statement "given the challenge α, my witness satisfies my constraints and my partial sum is s_i" —the challenge is *assumed*, not derived internally.

A natural first attempt is sequential hashing: Given commitments $\mathrm{com}_1, ..., \mathrm{com}_k$, compute:

$$\alpha = H(\cdots H(H(\mathrm{com}_1, \mathrm{com}_2), \mathrm{com}_3)\cdots, \mathrm{com}_k).$$

This fails for two reasons. It is inherently sequential, which negates the parallelism of trace splitting. And it is order-dependent: changing the processing order changes the challenge, so partial batches cannot be computed independently and merged.

ZisK instead uses *LtHash* [Tea25], a lattice-based multiset hash function whose defining property is homomorphism over multiset union:

$$\mathrm{LtHash}(S \cup T) = \mathrm{LtHash}(S) + \mathrm{LtHash}(T),$$

where addition is component-wise over an appropriate vector space. Because the group operation is commutative, the accumulated value does not depend on the order in which commitments are processed: any subset can be accumulated independently and the results summed, enabling full parallelism and hierarchical composition.

The construction is as follows. Each prover $\mathcal{P}_i$ first hashes its commitment $c_i = H(\mathrm{com}_i)$, which is then *embedded* into a long pseudorandom vector $v_i := v_i(c_i) \in \mathbb{Z}_p^n$ of sufficient length by iterative hashing. The global accumulator is simply the component-wise sum of all these vectors:

$$\mathrm{acc} = \sum_i v_i \in \mathbb{Z}_p^n,$$

and the common challenge α is read off from acc. Because addition in $\mathbb{Z}_p^n$ is commutative, the result does not depend on the order of summation: any subset of provers can compute their partial sum independently and all partial sums can be merged later by addition. Security —the guarantee that no two distinct multisets of commitments produce the same acc— reduces to the Short Integer Solution (SIS) [Ajt96] lattice problem, which is believed to remain hard even against quantum adversaries.

Correctness of the assumed challenge must be verified —otherwise a malicious prover could substitute any α to make the partial sums balance artificially. Since LtHash is homomorphic, this verification is embedded directly into the aggregation tree: each node carries a *partial accumulator* over the commitments in its subtree alongside its partial LogUp sum, and a parent node combines the two by addition. At the root, the fully accumulated value is checked against the claimed α, confirming that it was correctly derived from the entire set of commitments.

6.4. **Recursive Structure.**

Individual AIR proofs are combined in a fixed arity aggregation tree illustrated in Figure 5. Each aggregation node has a fixed amount of children, but the tree is not required to be balanced: it is built opportunistically as proofs complete, so a branch that finishes early can be aggregated immediately while others are still running. The shape of the tree therefore reflects the relative timing of the individual proofs, not a fixed structure decided in advance.

At every level of the tree, each node *verifies* the proofs produced by its children. In addition to proof verification, each node accumulates two quantities upward: the LogUp partial sums s_i and the LtHash challenge contributions $\boldsymbol{c}_i$. These accumulations are the mechanism by which the two global checks — bus balancing and challenge correctness— are reduced to arithmetic on public outputs at the root.

The challenge α is an input to the entire aggregation structure, not something derived inside it. Before proving starts, α is derived from all commitments via LtHash and broadcast to every leaf prover. Each leaf proof takes α as a *public input* and uses it to compute its partial LogUp sum s_i. The aggregation tree then accumulates both s and the LtHash contributions $\boldsymbol{c}$ upward, and the root receives α as a public input to verify that it matches what the tree actually accumulated.

The tree has three node types.

Leaf nodes are the raw proofs produced by individual AIR segment instances. Each leaf takes α as a public input and exposes as public outputs:

- Its *own public outputs* $\boldsymbol{o}_i \in \mathbb{F}^k$: values the AIR explicitly exposes, such as program outputs that must be visible to the final verifier.
- Its *partial LogUp sum* $s_i \in \mathbb{K}$: the final value of the LogUp accumulator, representing AIR i's net signed contribution to all bus interactions.
- Its *challenge contribution* $\boldsymbol{c}_i \in \mathbb{Z}_p^n$: the LtHash vector embedding of its commitment, representing AIR i's additive share of the global challenge accumulator.

Compressor nodes sit directly above the leaves. Each compressor takes a single leaf proof as input and converts it into a format that can be verified efficiently inside the recursive aggregation circuit. The compressor outputs a compressed proof with the same public outputs as its leaf.

Aggregation nodes each receive the compressed proofs of their two children, verify both, and accumulate the LogUp partial sums and the challenge contributions:

$$s_{\text{parent}} = s_L + s_R, \quad \boldsymbol{c}_{\text{parent}} = \boldsymbol{c}_L + \boldsymbol{c}_R,$$

outputting $\left(s_{\text{parent}}, \boldsymbol{c}_{\text{parent}}\right)$ for the next level. This propagates up the tree until a single pair reaches the root.

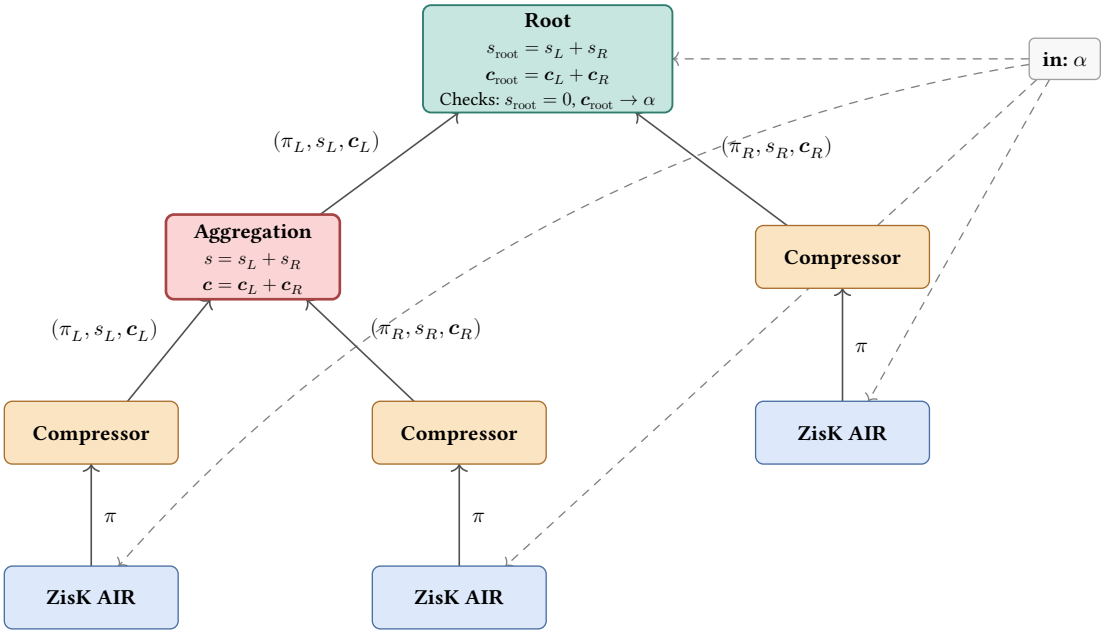


FIGURE 5. Example aggregation tree (binary, unbalanced). α is a public input broadcast to all leaf provers (dashed arrows) and to the root verifier. Solid arrows show proof propagation (bottom-up).

6.4.1. *Root Verification.*

At the root of the aggregation tree, two checks are performed simultaneously using the accumulated public outputs.

The first is *bus balancing*: the accumulated LogUp sum must be zero,

$$s_{\text{root}} = \sum_{i=1}^{k} s_i = 0,$$

which is the global bus-balancing condition from Definition 5 —every message sent on every bus has been received exactly once with matching values.

The second is *challenge correctness*: the accumulated LtHash vector $\boldsymbol{c}_{\text{root}} = \sum_i \boldsymbol{c}_i$ must be consistent with the challenge α assumed by all leaf proofs. Concretely, the root verifier checks that α equals the field element obtained by embedding $\boldsymbol{c}_{\text{root}}$ into $\mathbb{K}$, confirming that α was honestly derived from the full set of commitments and not chosen adversarially.

Both checks reduce to simple arithmetic on values propagated up the tree as public outputs, with no need to inspect any individual AIR trace. This hierarchical structure achieves a clean separation of responsibilities: leaf AIRs prove only local constraint satisfaction; compressor nodes convert proofs to a recursion-friendly format; aggregation nodes accumulate both running totals; and the root enforces global bus balancing and challenge integrity. Because every node can proceed as soon as its children complete, the entire proving pipeline is fully parallelizable —no single AIR needs to handle the entire computation, eliminating the bottleneck of a monolithic circuit.

Acknowledgments

This paper draws on the designs presented in the OpenVM [Ope25] and Nexus [Nex24] whitepapers, which served as important sources of inspiration. We also acknowledge RISC Zero [RIS23] as a pioneering zkVM and SP1 [Suc24] for advancing the practical deployment of RISC-V-based proving systems.

We extend our gratitude to the Polygon zkEVM and Plonky3 [Pol24] teams for their foundational work in zero-knowledge proving systems, as well as to the RISC-V community [RIS19] for providing a robust architecture that enables the zkVM model.

References

GWC19. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge, https://eprint.iacr.org/2019/953

Ben+18. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In: Chatzigiannakis, I., Kaklamanis, C., Marx, D., and Sannella, D. (eds.) ICALP 2018: 45th International Colloquium on Automata, Languages and Programming. pp. 14:1–14:17. Schloss Dagstuhl (2018a)

DP23. Diamond, B.E., Posen, J.: Succinct Arguments over Towers of Binary Fields, https://eprint.iacr.org/2023/1784

HLP24. Haböck, U., Levit, D., Papini, S.: Circle STARKs, https://eprint.iacr.org/2024/278

Bel+23. Bellés-Muñoz, M., Isabel, M., Muñoz-Tapia, J.L., Rubio, A., Baylina, J.: Circom: A Circuit Description Language for Building Zero-Knowledge Applications, (2023)

Ele22. Electric Coin Company: Halo2, https://github.com/zcash/halo2, (2022)

GPR21. Goldberg, L., Papini, S., Riabzev, M.: Cairo – a Turing-complete STARK-friendly CPU architecture, https://eprint.iacr.org/2021/1063

Lit24. Lita Foundation: Valida: a STARK-based virtual machine focusing on code reuse, https://github.com/valida-xyz/valida, (2024)

RIS19. RISC-V International: The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA. (2019)

RIS23. RISC Zero: RISC Zero zkVM: Scalable, transparent arguments of RISC-V integrity, (2023)

Suc24. Succinct Labs: SP1: A performant, open-source zkVM, (2024)

Nex24. Nexus: The Nexus zkVM, (2024)

Ben+18. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Scalable, transparent, and post-quantum secure computational integrity, https://eprint.iacr.org/2018/046

RIS24. RISC-V International: RISC-V ELF psABI Specification, (2024)

Hab22. Haböck, U.: Multivariate lookups based on logarithmic derivatives, https://eprint.iacr.org/2022/1530

FS87. Fiat, A., Shamir, A.: How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In: Odlyzko, A.M. (ed.) Advances in Cryptology – CRYPTO'86. pp. 186–194. Springer, Berlin, Heidelberg (1987)

Ope25. OpenVM Contributors: OpenVM Whitepaper, https://openvm.dev/whitepaper.pdf, (2025)

Hem+25. Hemo, T., Jue, K., Rabinovich, E., Roh, G., Rothblum, R.D.: Jagged Polynomial Commitments (or: How to Stack Multilinears), https://eprint.iacr.org/2025/917

Tea25. Team, Z.C.: Secure Challenge Derivation in ZisK, https://hackmd.io/SXKiWASTRu2QYgy7B-NGmg

Ajt96. Ajtai, M.: Generating Hard Instances of Lattice Problems (Extended Abstract). In: 28th Annual ACM Symposium on Theory of Computing. pp. 99–108. ACM Press (1996)

Pol24. Polygon: Plonky3, (2024)

Appendix

A. Arguments

In this section, we describe the distinct argument types used in ZisK's arithmetization.

Permutation Argument.

A *permutation argument* proves that two sequences of values are permutations of each other, i.e., they contain the same elements with the same multiplicities but possibly in different orders.

Definition (Permutation Argument). A *permutation argument* between two AIRs $\mathcal{A}_i$ and $\mathcal{A}_j$ with interaction values $\boldsymbol{e}_i$ and $\boldsymbol{e}_j$ and selectors s_i and s_j proves that:

$$\{\boldsymbol{e}_i(r) : r \in [0, n_i - 1], s_i(r) \neq 0\} = \left\{\boldsymbol{e}_j(r) : r \in \left[0, n_j - 1\right], s_j(r) \neq 0\right\}$$

as multisets.

Permutation arguments are used when we need to verify that data is reorganized but not modified.

Lookup Argument.

A *lookup argument* proves that every element in one sequence appears in another sequence (the "table"), without requiring the sequences to have the same length or the same multiplicity distribution.

Definition (Lookup Argument). A *lookup argument* between two AIRs $\mathcal{A}_{\text{query}}$ and $\mathcal{A}_{\text{table}}$ proves that for a sequence of query values $\boldsymbol{e}_{\text{query}}$ with selector s_{query} and a table $\boldsymbol{e}_{\text{table}}$ with selector s_{table}:

$$\forall r : s_{\text{query}}(r) \neq 0 \Rightarrow \exists r' : s_{\text{table}}(r') \neq 0 \wedge \boldsymbol{e}_{\text{query}}(r) = \boldsymbol{e}_{\text{table}}(r')$$

This is expressed with the bus using multiplicities: queries typically contribute $+1$ per active row, and table entries contribute with their lookup count.

Lookup arguments are fundamental for range checks, table-driven computations, and verifying that values belong to a predefined set. For example, verifying that a byte value is in the range $[0, 255]$ can be done via a lookup into a table containing all valid byte values.

Copy Argument.

A *copy argument* proves that designated cells across different parts of the witness hold equal values, establishing equivalence classes among witness cells.

Definition (Copy Argument). A *copy argument* proves that a set of witness cells forms equivalence classes where all cells in each class have the same value. Given cells $C = \{(i_1, r_1), ..., (i_m, r_m)\}$ partitioned into equivalence classes $\{C_1, ..., C_t\}$:

$$\forall (i, r), (i', r') \in C_\ell : e_i(r) = e_{i'}(r')$$

for each equivalence class C_ℓ. This is implemented as a permutation argument on the cells within each equivalence class.

Copy arguments are used to enforce consistency between AIRs (not necessarily distinct) that share intermediate values. For example, ensuring that the output of an ALU operation matches the value written to a register, or establishing complex wiring patterns where multiple cells must hold identical values.